DBMS LAB RECORD

(Ritika.S – 23011101114)

TABLE OF CONTENTS

SNO	TITLE	DATE	PAGE
1.	Problem Identification	20-12-2024	3
2.	Requirement Gathering	27-12-2024	4-6
3.	Design Using Entity Relationship (ER) Diagram	03-01-2025	7
4.	Design Using Unified Modelling Language (UML) Diagram	10-01-2025	8-10
5.	ER to Relational scheme mapping	17-01-2025	11
6.	Working with DDL and DML Commands	24-01-2025	12-16
7.	Working with Joins, Set operations and Aggregate Functions	31-01-2025	17-20
8.	Working with queries related to Projects	07-02-2025	21-34
9.	Working with PL/SQL Procedures	21-02-2025	35-42
10.	Working with PL/SQL Functions	28-02-2025	43-45
11.	Working with PL/SQL Cursors	07-03-2025	45-59
12.	Working with PL/SQL Triggers	21-03-2025	60-66
13.	Project Report	28-03-2025	67-74

1. PROBLEM IDENTIFICATION

The Pharmacy Management System (PMS) is designed to help users quickly find out where they can buy the medicines they need. The system uses a database to store important details like medicine names, available quantities, expiry dates, prices, and pharmacy locations.

The main goal of this system is to make it easy for people to find medicines in nearby pharmacies. Users can enter information such as the medicine name, quantity, and expiry date to search for pharmacies that have the medicine in stock. The system will then show a list of pharmacies nearby, sorted by distance, along with details like how much of the medicine is available, the price, and contact information.

By connecting different pharmacies in the area, the system helps users save time and effort in finding medicines. It also makes it easier for pharmacies to share their stock information, improving the overall process of buying medicine.

The system will use basic database management techniques to ensure all pharmacy data is stored correctly and retrieved easily. It will also use location-based methods to find the nearest pharmacies to the user's location, making the search faster and more accurate. In the end, this system will make it simpler and quicker for people to find the medicines they need.

2. REQUIREMENT GATHERING

Functional Requirements

1. Medicine Search: The system shall allow users to search for medicines based on name, quantity, and expiry date.
2. Pharmacy Listings: The system shall display a list of pharmacies with available medicines, sorted by distance.
3. Location Services: The system shall use geolocation to display pharmacies that are nearest to the user.
4. Pharmacy Information: The system shall show pharmacy details including the medicine's availability, price, contact number, and address.
5. Stock Update: The system shall allow pharmacy administrators to update stock quantities, prices, and expiry dates.
6. User Registration: Users and pharmacy admins shall be able to register and log into the system.
7. Admin Panel: The system shall provide an admin panel for pharmacy owners to manage their inventory.

External Interface Requirements

- User Interfaces

- Login Page: For user and pharmacy admin authentication.
- Search Page: Allows users to input search criteria (medicine name, quantity, expiry).

- Results Page: Displays a list of pharmacies with the availability of the searched medicine.
- Admin Dashboard: For pharmacy admins to update inventory.

- Hardware Interfaces

- Server: The application will be hosted on a cloud server that manages the data and handles multiple user requests.
- User Devices: The application will be accessible from desktops, laptops, and mobile devices.

- Software Interfaces

- Database: SQL-based database system for storing pharmacy and medicine information.
- GPS/Location Services: Integration with geolocation services for determining the proximity of pharmacies.

- Communications Interfaces

- Web-based communication: The system will use HTTP/HTTPS protocols for communication between the client-side application and the server.
- API Integration: The system will communicate with external APIs for geolocation services to fetch pharmacy proximity.

Non functional Requirements

- Performance Requirements

- The system should respond to a search query within 3 seconds.
- The system should handle at least 10,000 concurrent users.

- Safety Requirements

- The system should comply with data protection and privacy regulations, including GDPR if applicable.

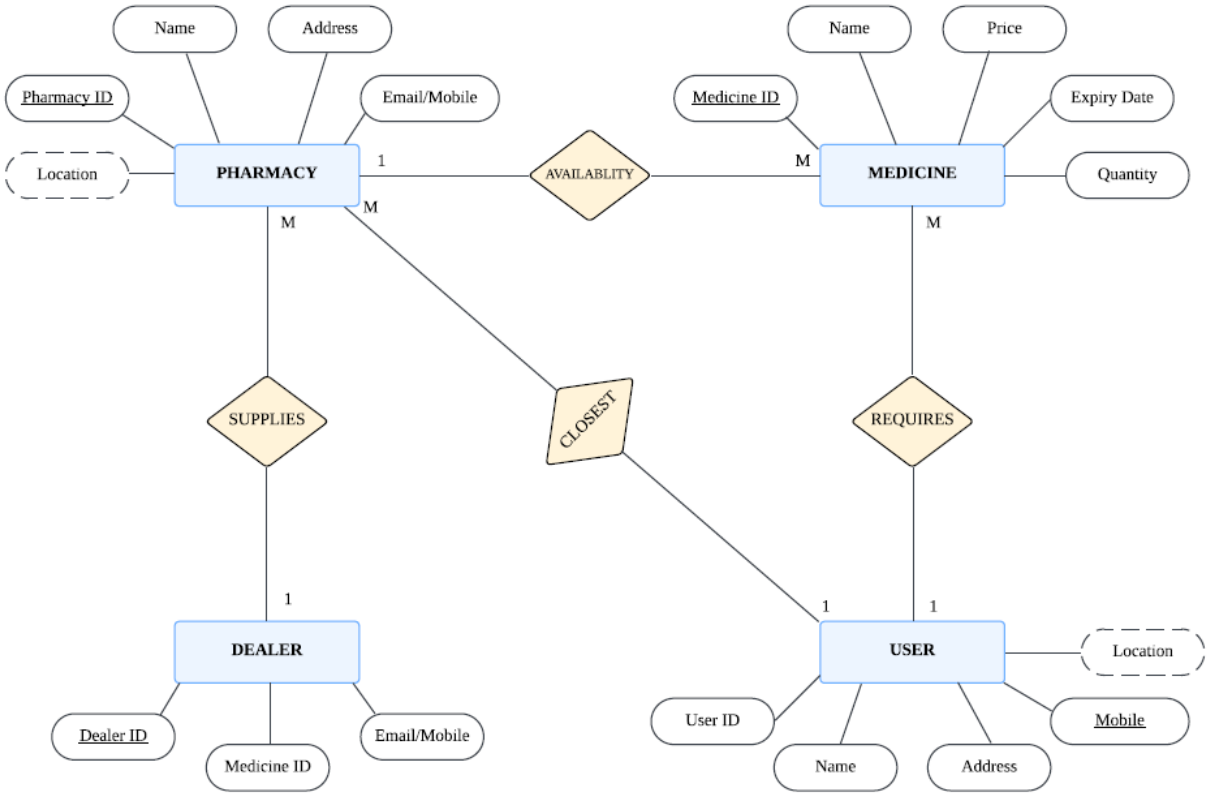
- Security Requirements

- The system shall use SSL/TLS encryption for secure communication.
- User credentials should be hashed and stored securely.
- User sessions shall time out after a period of inactivity.

- Software Quality Attributes

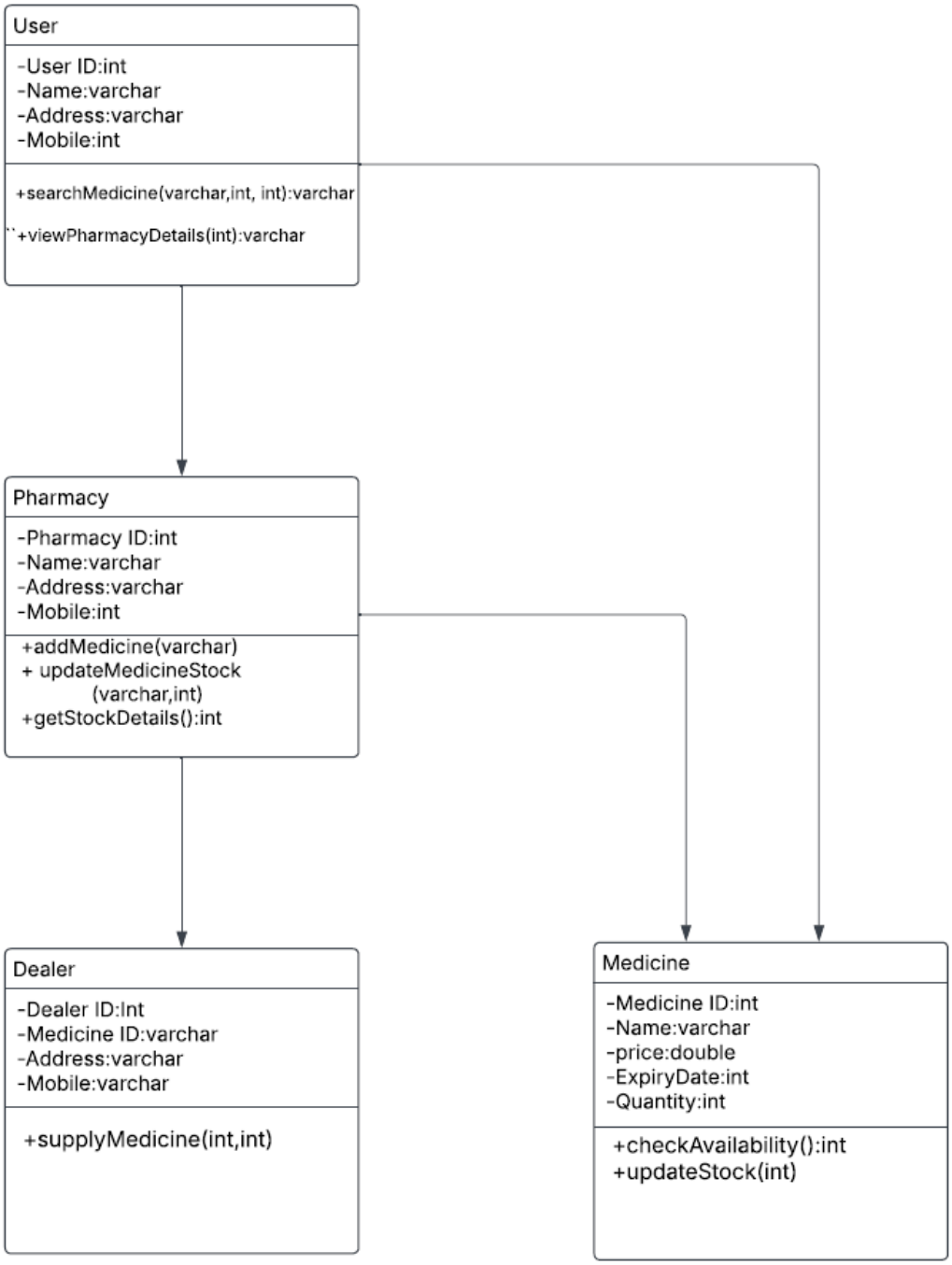
- Reliability: The system should be available 99.9% of the time, excluding maintenance periods.
- Usability: The system should be easy to navigate, with clear instructions for users and admins.
- Scalability: The system should be scalable to accommodate more users and pharmacies in the future.

3. ER DIAGRAM

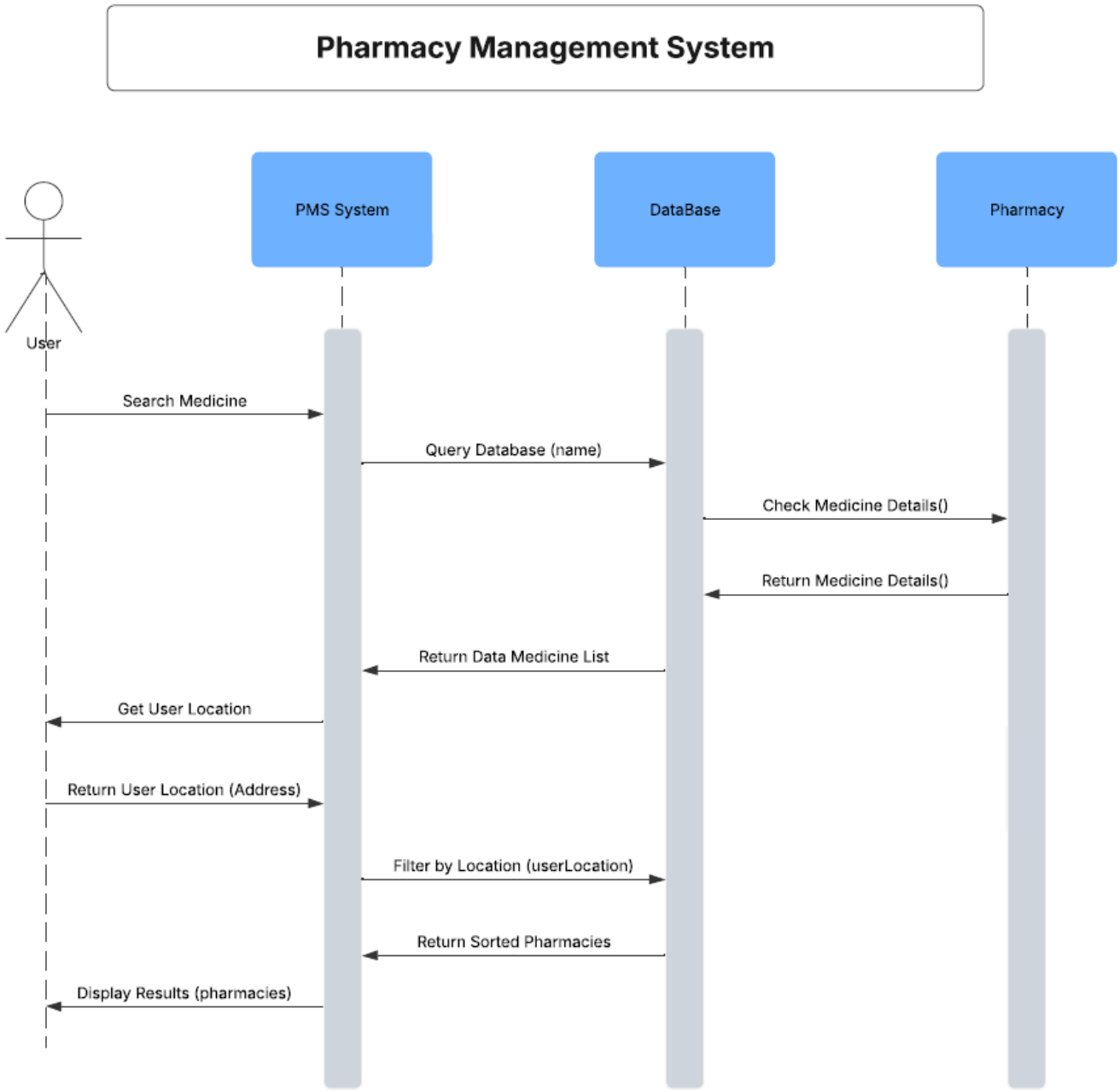

Name
Address
Pharmacy ID
Email/Mobile
Location
PHARMACY
1
M
M
AVAILABLITY
Name
Price
Medicine ID
Expiry Date
M
MEDICINE
Quantity
M
SUPPLIES
CLOSEST
REQUIRES
1
1
1
DEALER
USER
Location
Dealer ID
Medicine ID
Email/Mobile
User ID
Mobile
Name
Address

4. UML DIAGRAMS

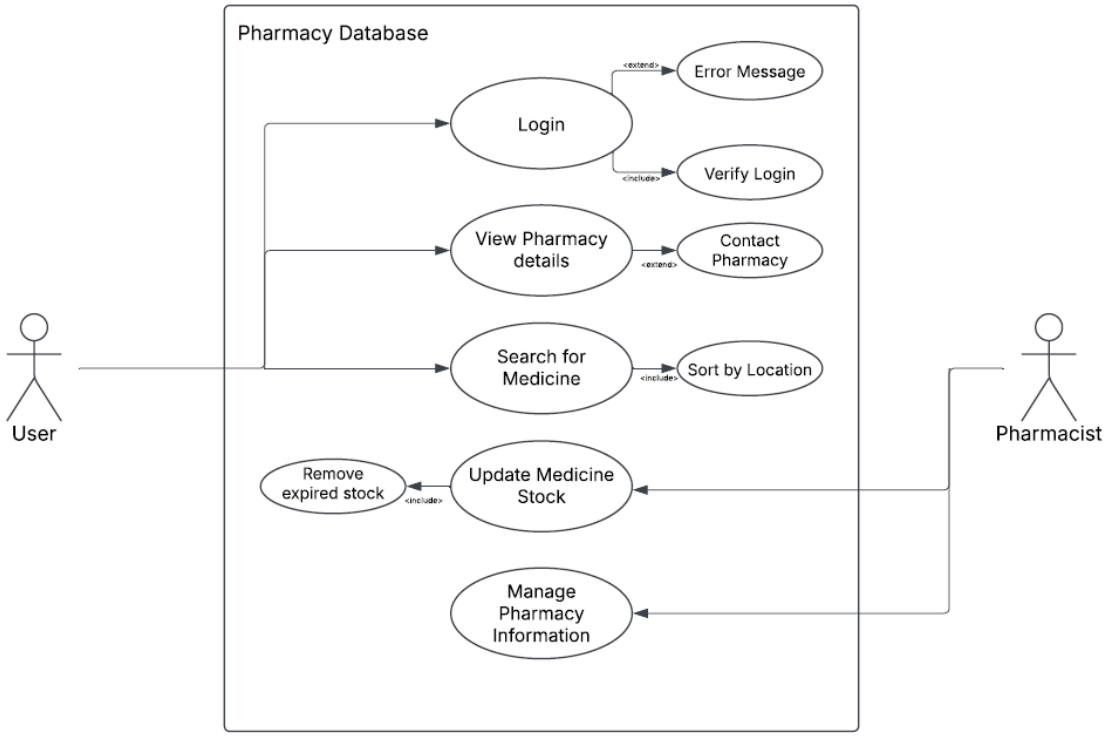
Class diagram:



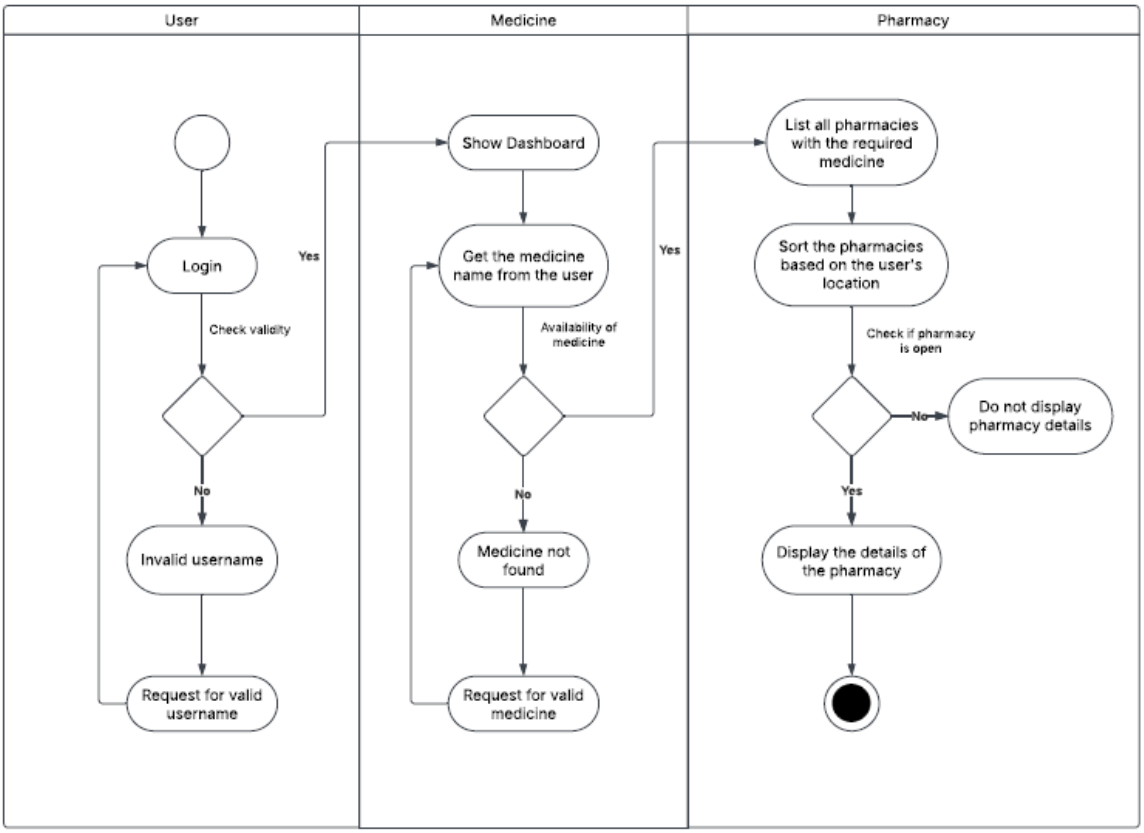
Sequence diagram:



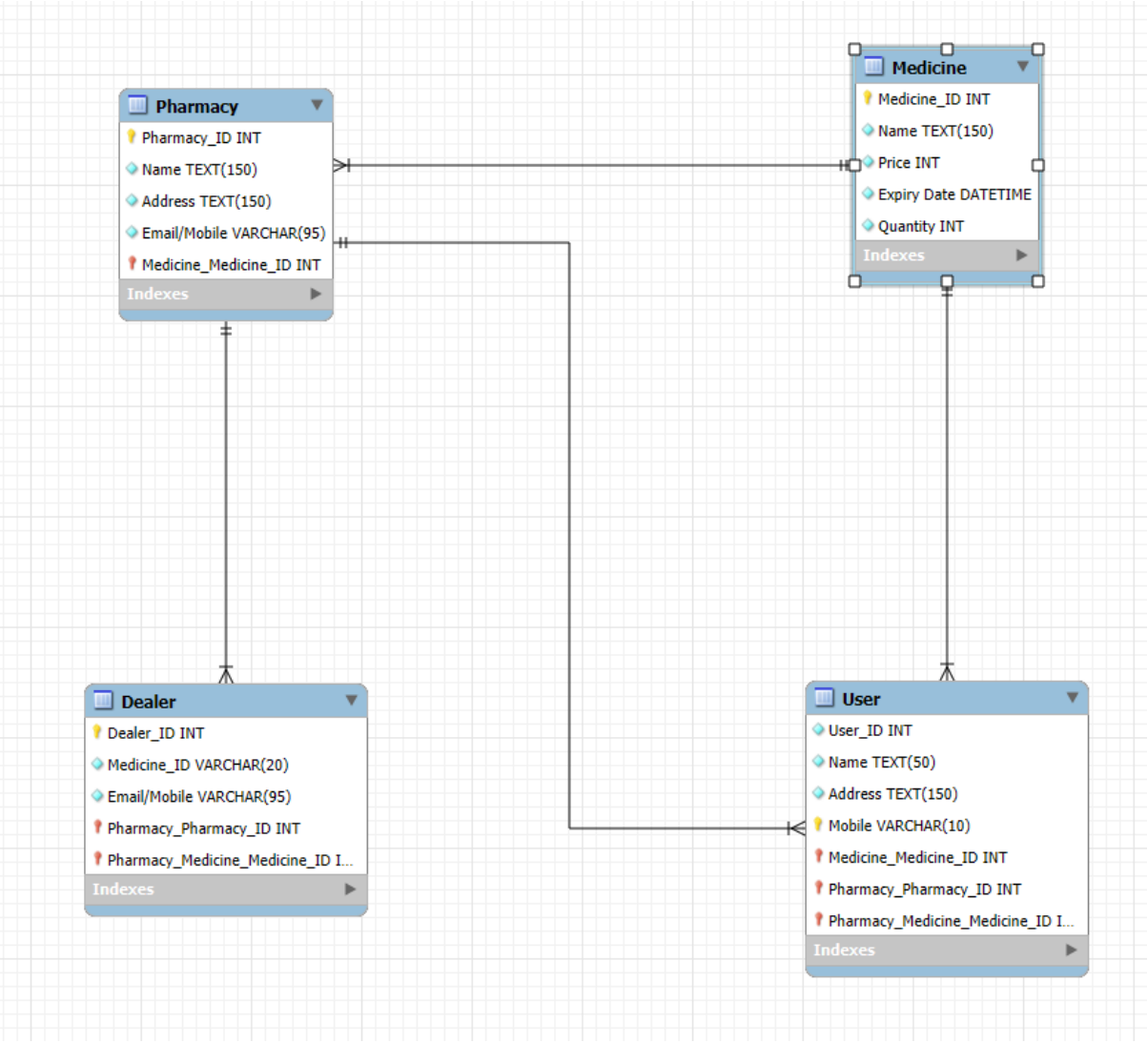
Use Case diagram:



Activity diagram:



5. ER to RELATIONAL SCHEMA MAPPING



6. WORKING WITH DDL AND DML COMMANDS

Data Definition Language (DDL):

- Creating a Table

```
mysql> USE lab;
Database changed
mysql> CREATE TABLE Employees (
    ->     EmpID INT PRIMARY KEY,
    ->     Name VARCHAR(50),
    ->     Age INT,
    ->     Department VARCHAR(50),
    ->     Salary FLOAT
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> SHOW TABLES;
+---------------+
| Tables_in_lab |
+---------------+
| employees     |
+---------------+
1 row in set (0.04 sec)
```

- Altering a Table

```
mysql> ALTER TABLE Employees ADD (Email VARCHAR(100));
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESC Employees;
+------------+--------------+------+-----+---------+-------+
| Field      | Type         | Null | Key | Default | Extra |
+------------+--------------+------+-----+---------+-------+
| EmpID      | int          | NO   | PRI | NULL    |       |
| Name       | varchar(50)  | YES  |     | NULL    |       |
| Age        | int          | YES  |     | NULL    |       |
| Department | varchar(50)  | YES  |     | NULL    |       |
| Salary     | float        | YES  |     | NULL    |       |
| Email      | varchar(100) | YES  |     | NULL    |       |
+------------+--------------+------+-----+---------+-------+
6 rows in set (0.02 sec)
```

- Modifying a Table

```
mysql> ALTER TABLE Employees MODIFY Age VARCHAR(3);
Query OK, 0 rows affected (0.10 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESC Employees;
+------------+--------------+------+-----+---------+-------+
| Field      | Type         | Null | Key | Default | Extra |
+------------+--------------+------+-----+---------+-------+
| EmpID      | int          | NO   | PRI | NULL    |       |
| Name       | varchar(50)  | YES  |     | NULL    |       |
| Age        | varchar(3)   | YES  |     | NULL    |       |
| Department | varchar(50)  | YES  |     | NULL    |       |
| Salary     | float        | YES  |     | NULL    |       |
| Email      | varchar(100) | YES  |     | NULL    |       |
+------------+--------------+------+-----+---------+-------+
6 rows in set (0.00 sec)
```

- Dropping a Column

```
mysql> ALTER TABLE Employees DROP COLUMN Email;
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESC Employees;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| EmpID      | int         | NO   | PRI | NULL    |       |
| Name       | varchar(50) | YES  |     | NULL    |       |
| Age        | varchar(3)  | YES  |     | NULL    |       |
| Department | varchar(50) | YES  |     | NULL    |       |
| Salary     | float       | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
```

- Dropping a Table

```
mysql> DROP TABLE Employees;
Query OK, 0 rows affected (0.03 sec)

mysql> SHOW TABLES;
Empty set (0.00 sec)
```

- Truncate a Table

```
mysql> TRUNCATE TABLE Employees;
Query OK, 0 rows affected (0.05 sec)

mysql> SELECT * FROM Employees;
Empty set (0.00 sec)
```

Data Manipulation Language (DML):

- Inserting into a Table

```
mysql> INSERT INTO Employees (EmpID, Name, Age, Department, Salary)
    -> VALUES (101, 'John Doe', 30, 'IT', 55000);
Query OK, 1 row affected (0.02 sec)

mysql> SELECT * FROM Employees;
+-------+----------+------+------------+--------+
| EmpID | Name     | Age  | Department | Salary |
+-------+----------+------+------------+--------+
|   101 | John Doe |   30 | IT         |  55000 |
+-------+----------+------+------------+--------+
1 row in set (0.00 sec)
```

- Updating a Table

```
mysql> UPDATE Employees
    -> SET Salary = 60000
    -> WHERE EmpID = 101;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Employees;
+-------+----------+------+------------+--------+
| EmpID | Name     | Age  | Department | Salary |
+-------+----------+------+------------+--------+
|   101 | John Doe |   30 | IT         |  60000 |
|   102 | Alice    |   28 | HR         |  48000 |
|   103 | Bob      |   35 | Finance    |  62000 |
+-------+----------+------+------------+--------+
3 rows in set (0.00 sec)
```

- Deleting from a Table

```
mysql> DELETE FROM Employees
    -> WHERE EmpID = 103;
Query OK, 1 row affected (0.02 sec)

mysql> SELECT * FROM Employees
    -> ;
+-------+----------+------+------------+--------+
| EmpID | Name     | Age  | Department | Salary |
+-------+----------+------+------------+--------+
|   101 | John Doe |   30 | IT         |  60000 |
|   102 | Alice    |   29 | Marketing  |  48000 |
+-------+----------+------+------------+--------+
2 rows in set (0.00 sec)
```

- Selecting from Table

```
mysql> SELECT * FROM Employees;
+-------+----------+------+------------+--------+
| EmpID | Name     | Age  | Department | Salary |
+-------+----------+------+------------+--------+
|   101 | John Doe |   30 | IT         |  60000 |
|   102 | Alice    |   29 | Marketing  |  48000 |
|   103 | Bob      |   35 | Finance    |  62000 |
+-------+----------+------+------------+--------+
3 rows in set (0.00 sec)
```

- Group by command

```
mysql> SELECT Department, AVG(Salary) AS Avg_Salary
    -> FROM Employees
    -> GROUP BY Department;
+------------+------------+
| Department | Avg_Salary |
+------------+------------+
| IT         |      60000 |
| Marketing  |      48000 |
| Finance    |      62000 |
+------------+------------+
3 rows in set (0.01 sec)
```

- Order by command

```
mysql> SELECT * FROM Employees
    -> ORDER BY Salary DESC;
+-------+----------+------+------------+--------+
| EmpID | Name     | Age  | Department | Salary |
+-------+----------+------+------------+--------+
|   103 | Bob      |   35 | Finance    |  62000 |
|   101 | John Doe |   30 | IT         |  60000 |
|   102 | Alice    |   29 | Marketing  |  48000 |
+-------+----------+------+------------+--------+
3 rows in set (0.00 sec)
```

- Where command

```
mysql> SELECT * FROM Employees
    -> WHERE Department = 'IT';
+-------+----------+------+------------+--------+
| EmpID | Name     | Age  | Department | Salary |
+-------+----------+------+------------+--------+
|   101 | John Doe |   30 | IT         |  60000 |
+-------+----------+------+------------+--------+
1 row in set (0.00 sec)
```

---

7. WORKING WITH JOINS, SET OPERATIONS AND AGGREGATE FUNCTIONS

Joins:

- Inner Join

```
mysql> SELECT Employees.EmpID, Employees.Name, Employees.Department, Departments.DeptName
    -> FROM Employees
    -> INNER JOIN Departments ON Employees.Department = Departments.DeptID;
+-------+---------+------------+-----------------+
| EmpID | Name    | Department | DeptName        |
+-------+---------+------------+-----------------+
|   101 | Alice   |          1 | IT Department   |
|   105 | Eve     |          1 | IT Department   |
|   102 | Bob     |          2 | Human Resources |
|   103 | Charlie |          3 | Finance Dept    |
|   106 | Frank   |          4 | Marketing       |
+-------+---------+------------+-----------------+
5 rows in set (0.00 sec)
```

- Left Join

```
mysql> SELECT Employees.EmpID, Employees.Name, Employees.Department, Departments.DeptName
    -> FROM Employees
    -> LEFT JOIN Departments ON Employees.Department = Departments.DeptID;
+-------+---------+------------+-----------------+
| EmpID | Name    | Department | DeptName        |
+-------+---------+------------+-----------------+
|   101 | Alice   |          1 | IT Department   |
|   102 | Bob     |          2 | Human Resources |
|   103 | Charlie |          3 | Finance Dept    |
|   104 | David   |       NULL | NULL            |
|   105 | Eve     |          1 | IT Department   |
|   106 | Frank   |          4 | Marketing       |
+-------+---------+------------+-----------------+
```

- Right Join

```
mysql> SELECT Employees.EmpID, Employees.Name, Employees.Department, Departments.DeptName
    -> FROM Employees
    -> RIGHT JOIN Departments ON Employees.Department = Departments.DeptID;
+-------+---------+------------+-----------------+
| EmpID | Name    | Department | DeptName        |
+-------+---------+------------+-----------------+
|   101 | Alice   |          1 | IT Department   |
|   105 | Eve     |          1 | IT Department   |
|   102 | Bob     |          2 | Human Resources |
|   103 | Charlie |          3 | Finance Dept    |
|   106 | Frank   |          4 | Marketing       |
|  NULL | NULL    |       NULL | Sales           |
+-------+---------+------------+-----------------+
6 rows in set (0.00 sec)
```

- Cross Join

```
mysql> SELECT Employees.Name, Departments.DeptName
    -> FROM Employees
    -> CROSS JOIN Departments;
+---------+-----------------+
| Name    | DeptName        |
+---------+-----------------+
| Alice   | Sales           |
| Alice   | Marketing       |
| Alice   | Finance Dept    |
| Alice   | Human Resources |
| Alice   | IT Department   |
| Bob     | Sales           |
| Bob     | Marketing       |
| Bob     | Finance Dept    |
| Bob     | Human Resources |
| Bob     | IT Department   |
| Charlie | Sales           |
| Charlie | Marketing       |
| Charlie | Finance Dept    |
| Charlie | Human Resources |
| Charlie | IT Department   |
| David   | Sales           |
| David   | Marketing       |
| David   | Finance Dept    |
| David   | Human Resources |
| David   | IT Department   |
| Eve     | Sales           |
| Eve     | Marketing       |
| Eve     | Finance Dept    |
| Eve     | Human Resources |
| Eve     | IT Department   |
| Frank   | Sales           |
| Frank   | Marketing       |
| Frank   | Finance Dept    |
| Frank   | Human Resources |
| Frank   | IT Department   |
+---------+-----------------+
30 rows in set (0.00 sec)
```

- Self Join

```
mysql> SELECT A.Name AS Employee1, B.Name AS Employee2, A.Department
    -> FROM Employees A
    -> JOIN Employees B ON A.Department = B.Department
    -> WHERE A.EmpID <> B.EmpID;
+-----------+-----------+------------+
| Employee1 | Employee2 | Department |
+-----------+-----------+------------+
| Alice     | Eve       |          1 |
| Eve       | Alice     |          1 |
+-----------+-----------+------------+
2 rows in set (0.00 sec)
```

Set Operations:

- Union

```
mysql> SELECT Department FROM Employees
    -> UNION
    -> SELECT DeptName FROM Departments;
+-----------------+
| Department      |
+-----------------+
| NULL            |
| 1               |
| 2               |
| 3               |
| 4               |
| IT Department   |
| Human Resources |
| Finance Dept    |
| Marketing       |
| Sales           |
+-----------------+
10 rows in set (0.00 sec)
```

- Union all

```
mysql> SELECT Department FROM Employees
    -> UNION ALL
    -> SELECT DeptName FROM Departments;
+-----------------+
| Department      |
+-----------------+
| NULL            |
| 1               |
| 1               |
| 2               |
| 3               |
| 4               |
| IT Department   |
| Human Resources |
| Finance Dept    |
| Marketing       |
| Sales           |
+-----------------+
11 rows in set (0.00 sec)
```

Aggregate functions:

- Sum()

```
mysql> SELECT SUM(Salary) AS TotalSalary FROM Employees;
+-------------+
| TotalSalary |
+-------------+
|   362000.00 |
+-------------+
1 row in set (0.00 sec)
```

- Avg()

```
mysql> SELECT AVG(Salary) AS AvgSalary FROM Employees;
+--------------+
| AvgSalary    |
+--------------+
| 60333.333333 |
+--------------+
1 row in set (0.00 sec)
```

- Count()

```
mysql> SELECT COUNT(*) AS EmployeeCount FROM Employees;
+---------------+
| EmployeeCount |
+---------------+
|             6 |
+---------------+
1 row in set (0.00 sec)
```

- Max() and min()

```
mysql> SELECT MAX(Salary) AS HighestSalary, MIN(Salary) AS LowestSalary FROM Employees;
+--------------+--------------+
| HighestSalary | LowestSalary |
+--------------+--------------+
|      70000.00 |     50000.00 |
+--------------+--------------+
1 row in set (0.00 sec)
```

---

8. QUERIES RELATED TO THE PROJECT

1. What is the stock quantity of a specific medicine in each pharmacy?

```
SELECT p.pharmacy_name, s.quantity
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Medicine_Name';
```

---

2. Which pharmacies have a specific medicine in stock?

```
SELECT DISTINCT p.pharmacy_name
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Medicine_Name' AND
s.quantity > 0;
```

---

3. What are the prices of a specific medicine at all pharmacies?

```
SELECT p.pharmacy_name, s.price
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Medicine_Name';
```

---

4. Find the pharmacies nearest to a user's location (assuming lat/long data is available).

```
SELECT p.pharmacy_name,
      ( 6371 * acos( cos( radians(user_lat) ) * cos(
radians(p.latitude) ) *
      cos( radians(p.longitude) - radians(user_long) ) + sin(
radians(user_lat) ) *
      sin( radians(p.latitude) ) ) ) AS distance
FROM Pharmacy p
ORDER BY distance ASC
LIMIT 5;
```

---

5. What medicines are available in pharmacies within a certain distance from the user?

```
SELECT DISTINCT m.medicine_name
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE ( 6371 * acos( cos( radians(user_lat) ) * cos(
radians(p.latitude) ) *
      cos( radians(p.longitude) - radians(user_long) ) + sin(
radians(user_lat) ) *
      sin( radians(p.latitude) ) ) ) < user_distance;
```

---

6. Which pharmacies are currently open?

```
SELECT pharmacy_name
FROM Pharmacy
WHERE CURTIME() BETWEEN opening_time AND
closing_time;
```

---

7. What are the expiry dates for a medicine in stock at a pharmacy?

```
SELECT p.pharmacy_name, m.medicine_name, s.expiry_date
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Medicine_Name';
```

---

8. Show me the top 5 pharmacies with the most stock for a specific medicine.

```
SELECT p.pharmacy_name, s.quantity
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Medicine_Name'
ORDER BY s.quantity DESC
LIMIT 5;
```

---

9. What medicines are available in a pharmacy based on a user's health condition?

```
SELECT DISTINCT m.medicine_name
FROM Medicine m
JOIN Condition_Medicine cm ON m.medicine_id =
cm.medicine_id
WHERE cm.condition_name = 'User_Condition';
```

---

10. Which pharmacies sell a medicine within a specified price range?

```
SELECT p.pharmacy_name, s.price
```

```
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Medicine_Name' AND s.price
BETWEEN min_price AND max_price;
```

---

11. How many different types of medicines are available in a pharmacy?

```
SELECT p.pharmacy_name, COUNT(DISTINCT
s.medicine_id) AS medicine_count
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
GROUP BY p.pharmacy_name;
```

---

12. Which pharmacies offer the medicine 'Zyrtec' in stock?

```
SELECT p.pharmacy_name
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Zyrtec' AND s.quantity > 0;
```

---

13. Which pharmacy has the lowest price for 'Aspirin'?

```
SELECT p.pharmacy_name, s.price
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Aspirin'
ORDER BY s.price ASC
```

```
LIMIT 1;
```

---

14. Which pharmacies have 'Penicillin' in stock but at a price above $50?

```
SELECT p.pharmacy_name, s.price
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Penicillin' AND s.quantity > 0
AND s.price > 50;
```

---

15. Which medicines are about to expire in the next 7 days?

```
SELECT DISTINCT m.medicine_name
FROM Stock s
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE s.expiry_date BETWEEN CURDATE() AND
DATE_ADD(CURDATE(), INTERVAL 7 DAY);
```

---

16. Which pharmacies carry 'Penicillin' at a price between $20 and $30?

```
SELECT p.pharmacy_name
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Penicillin' AND s.price
BETWEEN 20 AND 30;
```

---

17. How many pharmacies are currently out of stock of 'Paracetamol'?

```
SELECT COUNT(DISTINCT p.pharmacy_id) AS
out_of_stock_count
FROM Pharmacy p
LEFT JOIN Stock s ON p.pharmacy_id = s.pharmacy_id
LEFT JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Paracetamol' AND (s.quantity
= 0 OR s.quantity IS NULL);
```

---

18. What are the opening hours of pharmacies that sell 'Ibuprofen'?

```
SELECT p.pharmacy_name, p.opening_time, p.closing_time
FROM Pharmacy p
JOIN Stock s ON p.pharmacy_id = s.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Ibuprofen' AND s.quantity > 0;
```

---

19. What medicines are in stock and have a price below $15?

```
SELECT DISTINCT m.medicine_name
FROM Stock s
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE s.quantity > 0 AND s.price < 15;
```

---

21. Which pharmacies in 'Los Angeles' offer 'Prescription Drugs'?

```
SELECT DISTINCT p.pharmacy_name
```

```
FROM Pharmacy p
JOIN Stock s ON p.pharmacy_id = s.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE p.city = 'Los Angeles' AND m.category =
'Prescription Drug' AND s.quantity > 0;
```

---

22. What pharmacies have 'Amoxicillin' with a shelf life of more than 6 months?

```
SELECT p.pharmacy_name
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Amoxicillin'
AND DATEDIFF(s.expiry_date, CURDATE()) > 180;
```

---

23. How many pharmacies have multiple brands of a certain medicine in stock?

```
SELECT COUNT(DISTINCT s.pharmacy_id) AS
pharmacy_count
FROM Stock s
JOIN Medicine m ON s.medicine_id = m.medicine_id
WHERE m.generic_name = 'Generic_Name'
GROUP BY s.pharmacy_id
```

```
HAVING COUNT(DISTINCT m.brand) > 1;
```

---

24. Which dealers supply a specific medicine to a pharmacy?

```
SELECT DISTINCT d.dealer_name
FROM Dealer d
JOIN Supply sp ON d.dealer_id = sp.dealer_id
JOIN Medicine m ON sp.medicine_id = m.medicine_id
WHERE m.medicine_name = 'Medicine_Name';
```

---

25. How many pharmacies receive supplies from the same dealer?

```
SELECT d.dealer_name, COUNT(DISTINCT
sp.pharmacy_id) AS pharmacy_count
FROM Dealer d
JOIN Supply sp ON d.dealer_id = sp.dealer_id
GROUP BY d.dealer_name;
```

---

26. How many pharmacies are located in a specific area?

```
SELECT COUNT(*) AS pharmacy_count
FROM Pharmacy
WHERE city = 'City_Name';
```

---

27. How many unique users are linked to pharmacies in a specific city?

```
SELECT COUNT(DISTINCT u.user_id) AS unique_users
FROM Users u
JOIN Pharmacy p ON u.pharmacy_id = p.pharmacy_id
WHERE p.city = 'City_Name';
```

---

28. How many medicines have been marked as unavailable in all pharmacies?

```
SELECT COUNT(DISTINCT m.medicine_id) AS unavailable_medicines
FROM Medicine m
LEFT JOIN Stock s ON m.medicine_id = s.medicine_id
WHERE s.medicine_id IS NULL OR s.quantity = 0;
```

---

29. How many pharmacies stock only one type of medicine?

```
SELECT COUNT(p.pharmacy_id)
FROM (SELECT s.pharmacy_id, COUNT(DISTINCT s.medicine_id) AS medicine_count
    FROM Stock s
    GROUP BY s.pharmacy_id
```

```
    HAVING medicine_count = 1) AS
single_medicine_pharmacies;
```

---

30. Which dealers supply the highest number of pharmacies?

```
SELECT d.dealer_name, COUNT(DISTINCT
sp.pharmacy_id) AS pharmacy_count
FROM Dealer d
JOIN Supply sp ON d.dealer_id = sp.dealer_id
GROUP BY d.dealer_name
ORDER BY pharmacy_count DESC
LIMIT 1;
```

---

31. What is the most frequently purchased medicine in a specific city?

```
SELECT m.medicine_name, COUNT(*) AS purchase_count
FROM Orders o
JOIN Medicine m ON o.medicine_id = m.medicine_id
JOIN Pharmacy p ON o.pharmacy_id = p.pharmacy_id
WHERE p.city = 'City_Name'
GROUP BY m.medicine_name
ORDER BY purchase_count DESC
LIMIT 1;
```

---

32. Which pharmacy has the highest total stock value?

```
SELECT p.pharmacy_name, SUM(s.quantity * s.price) AS
total_stock_value
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
GROUP BY p.pharmacy_name
ORDER BY total_stock_value DESC
LIMIT 1;
```

---

33. What is the average price of all medicine in a specific pharmacy?

```
SELECT p.pharmacy_name, AVG(s.price) AS avg_price
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
WHERE p.pharmacy_name = 'Pharmacy_Name'
GROUP BY p.pharmacy_name;
```

---

34. How many customers have purchased medicine from a specific pharmacy?

```
SELECT COUNT(DISTINCT o.customer_id) AS
customer_count
FROM Orders o
```

```
JOIN Pharmacy p ON o.pharmacy_id = p.pharmacy_id
WHERE p.pharmacy_name = 'Pharmacy_Name';
```

---

35. Which dealer has supplied the largest quantity of medicine in the last year?

```
SELECT d.dealer_name, SUM(sp.quantity_supplied) AS total_quantity
FROM Dealer d
JOIN Supply sp ON d.dealer_id = sp.dealer_id
WHERE sp.supply_date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)
GROUP BY d.dealer_name
ORDER BY total_quantity DESC
LIMIT 1;
```

---

36. Which customers have purchased medicines from more than one pharmacy?

```
SELECT o.customer_id, COUNT(DISTINCT o.pharmacy_id) AS pharmacy_count
FROM Orders o
GROUP BY o.customer_id
HAVING pharmacy_count > 1;
```

---

37. Which pharmacies have the least variety of medicines available?

```
SELECT p.pharmacy_name, COUNT(DISTINCT s.medicine_id) AS medicine_count
FROM Stock s
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
GROUP BY p.pharmacy_name
ORDER BY medicine_count ASC
LIMIT 5;
```

---

38. How many medicines are currently in stock but will expire within the next month?

```
SELECT COUNT(DISTINCT s.medicine_id) AS expiring_medicines
FROM Stock s
WHERE s.quantity > 0 AND s.expiry_date BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 30 DAY);
```

---

39. Which pharmacies have the highest difference in medicine prices compared to the average market price?

```
SELECT p.pharmacy_name, m.medicine_name, ABS(s.price - avg_prices.avg_price) AS price_difference
FROM Stock s
```

```
JOIN Pharmacy p ON s.pharmacy_id = p.pharmacy_id
JOIN Medicine m ON s.medicine_id = m.medicine_id
JOIN (SELECT medicine_id, AVG(price) AS avg_price
FROM Stock GROUP BY medicine_id) avg_prices
ON s.medicine_id = avg_prices.medicine_id
ORDER BY price_difference DESC
LIMIT 5;
```

---

40. Which medicines have never been purchased by any customer?

```
SELECT m.medicine_name
FROM Medicine m
LEFT JOIN Orders o ON m.medicine_id = o.medicine_id
WHERE o.medicine_id IS NULL;
```

---

9. WORKING WITH PL/SQL PROCEDURES

1. Write a PL/SQL program to check if the given number is prime or not

Code:

```
DELIMITER //

CREATE PROCEDURE Check_Prime(IN num INT)
BEGIN
   DECLARE i INT DEFAULT 2;
   DECLARE is_prime BOOLEAN DEFAULT TRUE;

   IF num < 2 THEN
      SET is_prime = FALSE;
   ELSE
      WHILE i <= SQRT(num) DO
         IF num MOD i = 0 THEN
            SET is_prime = FALSE;
            SET i = num;
         END IF;
         SET i = i + 1;
      END WHILE;
   END IF;
```

```
    IF is_prime THEN
        SELECT 'Prime' AS Result;
    ELSE
        SELECT 'Not Prime' AS Result;
    END IF;
END //

DELIMITER ;
CALL Check_Prime(17);
```

Output:

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE Check_Prime(IN num INT)
    -> BEGIN
    ->     DECLARE i INT DEFAULT 2;
    ->     DECLARE is_prime BOOLEAN DEFAULT TRUE;
    ->
    ->     IF num < 2 THEN
    ->         SET is_prime = FALSE;
    ->     ELSE
    ->         WHILE i <= SQRT(num) DO
    ->             IF num MOD i = 0 THEN
    ->                 SET is_prime = FALSE;
    ->                 SET i = num;
    ->             END IF;
    ->             SET i = i + 1;
    ->         END WHILE;
    ->     END IF;
    ->
    ->     IF is_prime THEN
    ->         SELECT 'Prime' AS Result;
    ->     ELSE
    ->         SELECT 'Not Prime' AS Result;
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL Check_Prime(17);
+--------+
| Result |
+--------+
| Prime  |
+--------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

2. Write a PL/SQL program to display factorial of a given number

Code:

```
DELIMITER //

CREATE PROCEDURE Calculate_Factorial(IN num INT)
BEGIN
    DECLARE fact BIGINT DEFAULT 1;
    DECLARE i INT DEFAULT 1;

    IF num < 0 THEN
        SELECT 'Invalid input' AS Result;
    ELSE
        WHILE i <= num DO
            SET fact = fact * i;
            SET i = i + 1;
        END WHILE;
        SELECT fact AS Factorial;
    END IF;
END //

DELIMITER ;
CALL Calculate_Factorial(5);
```

Output:

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE Calculate_Factorial(IN num INT)
    -> BEGIN
    ->     DECLARE fact BIGINT DEFAULT 1;
    ->     DECLARE i INT DEFAULT 1;
    ->
    ->     IF num < 0 THEN
    ->         SELECT 'Invalid input' AS Result;
    ->     ELSE
    ->         WHILE i <= num DO
    ->             SET fact = fact * i;
    ->             SET i = i + 1;
    ->         END WHILE;
    ->         SELECT fact AS Factorial;
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL Calculate_Factorial(5);
+-----------+
| Factorial |
+-----------+
|       120 |
+-----------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

---

3. Write a PL/SQL program to generate the Fibonacci Series

Code:

```
DELIMITER //


CREATE PROCEDURE Generate_Fibonacci(IN n INT)
BEGIN
    DECLARE a INT DEFAULT 0;
    DECLARE b INT DEFAULT 1;
```

```
    DECLARE c INT;
    DECLARE i INT DEFAULT 3;

    IF n < 1 THEN
        SELECT 'Invalid input' AS Output;
    ELSE
        SELECT a AS FibonacciNumber;
        IF n > 1 THEN
            SELECT b AS FibonacciNumber;
        END IF;

        WHILE i <= n DO
            SET c = a + b;
            SELECT c AS FibonacciNumber;
            SET a = b;
            SET b = c;
            SET i = i + 1;
        END WHILE;
    END IF;
END //

DELIMITER ;
```

CALL Generate_Fibonacci(10);

Output:

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE Generate_Fibonacci(IN n INT)
    -> BEGIN
    ->     DECLARE a INT DEFAULT 0;
    ->     DECLARE b INT DEFAULT 1;
    ->     DECLARE c INT;
    ->     DECLARE i INT DEFAULT 3;
    ->
    ->     IF n < 1 THEN
    ->         SELECT 'Invalid input' AS Output;
    ->     ELSE
    ->         SELECT a AS FibonacciNumber;
    ->         IF n > 1 THEN
    ->             SELECT b AS FibonacciNumber;
    ->         END IF;
    ->
    ->         WHILE i <= n DO
    ->             SET c = a + b;
    ->             SELECT c AS FibonacciNumber;
    ->             SET a = b;
    ->             SET b = c;
    ->             SET i = i + 1;
    ->         END WHILE;
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql>
```

```
mysql> CALL Generate_Fibonacci(10);
+-----------------+
| FibonacciNumber |
+-----------------+
|               0 |
+-----------------+
1 row in set (0.00 sec)

+-----------------+
| FibonacciNumber |
+-----------------+
|               1 |
+-----------------+
1 row in set (0.01 sec)

+-----------------+
| FibonacciNumber |
+-----------------+
|               1 |
+-----------------+
1 row in set (0.01 sec)

+-----------------+
| FibonacciNumber |
+-----------------+
|               2 |
+-----------------+
1 row in set (0.01 sec)

+-----------------+
| FibonacciNumber |
+-----------------+
|               3 |
+-----------------+
1 row in set (0.02 sec)
```

```
+-----------------+
| FibonacciNumber |
+-----------------+
|               5 |
+-----------------+
1 row in set (0.02 sec)

+-----------------+
| FibonacciNumber |
+-----------------+
|               8 |
+-----------------+
1 row in set (0.04 sec)

+-----------------+
| FibonacciNumber |
+-----------------+
|              13 |
+-----------------+
1 row in set (0.04 sec)

+-----------------+
| FibonacciNumber |
+-----------------+
|              21 |
+-----------------+
1 row in set (0.04 sec)

+-----------------+
| FibonacciNumber |
+-----------------+
|              34 |
+-----------------+
1 row in set (0.05 sec)

Query OK, 0 rows affected (0.05 sec)

mysql> |
```

---

5. Write a PL/SQL program to check if the given number is palindrome or not

Code:

```
DELIMITER //

CREATE PROCEDURE Check_Palindrome(IN num INT)
BEGIN
    DECLARE temp INT;
    DECLARE rev INT DEFAULT 0;
    DECLARE rem INT;

    IF num < 0 THEN
        SELECT 'Invalid input' AS Result;
    ELSE
        SET temp = num;

        WHILE temp > 0 DO
            SET rem = MOD(temp, 10);
            SET rev = rev * 10 + rem;
            SET temp = FLOOR(temp / 10);
        END WHILE;

        IF num = rev THEN
            SELECT 'Palindrome' AS Result;
        ELSE
            SELECT 'Not Palindrome' AS Result;
```

```
        END IF;
    END IF;
END //

DELIMITER ;

CALL Check_Palindrome(121);
```

Output:

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE Check_Palindrome(IN num INT)
    -> BEGIN
    ->     DECLARE temp INT;
    ->     DECLARE rev INT DEFAULT 0;
    ->     DECLARE rem INT;
    ->
    ->     IF num < 0 THEN
    ->         SELECT 'Invalid input' AS Result;
    ->     ELSE
    ->         SET temp = num;
    ->
    ->         WHILE temp > 0 DO
    ->             SET rem = MOD(temp, 10);
    ->             SET rev = rev * 10 + rem;
    ->             SET temp = FLOOR(temp / 10);
    ->         END WHILE;
    ->
    ->         IF num = rev THEN
    ->             SELECT 'Palindrome' AS Result;
    ->         ELSE
    ->             SELECT 'Not Palindrome' AS Result;
    ->         END IF;
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL Check_Palindrome(121);
+------------+
| Result     |
+------------+
| Palindrome |
+------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

---

10. WORKING WITH PL/SQL FUNCTIONS

1.Write a PL/SQL program to reverse a given number

Code:

```
DELIMITER //

CREATE FUNCTION Reverse_Number(num INT)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
   DECLARE rev INT DEFAULT 0;
   DECLARE digit INT;
   DECLARE temp INT;

   SET temp = ABS(num);

   WHILE temp > 0 DO
      SET digit = temp MOD 10;
      SET rev = rev * 10 + digit;
      SET temp = temp DIV 10;
   END WHILE;
```

```
    IF num < 0 THEN
        SET rev = rev * -1;
    END IF;

    RETURN rev;
END //

DELIMITER ;
SELECT Reverse_Number(12345) AS ReversedNumber;
```

Output:

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION Reverse_Number(num INT) RETURNS INT
    -> DETERMINISTIC
    -> NO SQL
    -> BEGIN
    ->     DECLARE rev INT DEFAULT 0;
    ->     DECLARE digit INT;
    ->     DECLARE temp INT;
    ->
    ->     SET temp = ABS(num);
    ->
    ->     WHILE temp > 0 DO
    ->         SET digit = temp MOD 10;
    ->         SET rev = rev * 10 + digit;
    ->         SET temp = temp DIV 10;
    ->     END WHILE;
    ->
    ->     IF num < 0 THEN
    ->         SET rev = rev * -1;
    ->     END IF;
    ->
    ->     RETURN rev;
    -> END //
ERROR 1304 (42000): FUNCTION Reverse_Number already exists
mysql>
mysql> DELIMITER ;
mysql>
mysql> SELECT Reverse_Number(12345) AS ReversedNumber;
+----------------+
| ReversedNumber |
+----------------+
|          54321 |
+----------------+
1 row in set (0.00 sec)
```

11. WORKING WITH CURSORS

Creating required tables:

```
CREATE DATABASE db2;
USE db2;

CREATE TABLE employee(
emp_id INT PRIMARY KEY,
emp_name VARCHAR(25),
department VARCHAR(15),
salary INT
);

INSERT INTO employee VALUES(1, 'Sam', 'Engineering', 45000);
INSERT INTO employee VALUES(2, 'Ram', 'HR', 40000);
INSERT INTO employee VALUES(3, 'Pam', 'IT', 55000);
INSERT INTO employee VALUES(4, 'John', 'Engineering', 65000);
INSERT INTO employee VALUES(5, 'Mike', 'IT', 48000);
INSERT INTO employee VALUES(6, 'Alice', 'HR', 42000);
INSERT INTO employee VALUES(7, 'Bob', 'Engineering', 50000);
INSERT INTO employee VALUES(8, 'Charlie', 'IT', 60000);
```

```
INSERT INTO employee VALUES(9, 'David', 'HR', 39000);
INSERT INTO employee VALUES(10, 'Lewis', 'Engineering',
70000);
```

1. **Fetch and display employee details using a cursor.**

```
DELIMITER //
CREATE PROCEDURE print_details()
BEGIN

DECLARE cur_emp_id INT;
   DECLARE cur_emp_name VARCHAR(25);
   DECLARE cur_department VARCHAR(15);
   DECLARE cur_salary INT;
   DECLARE done INT DEFAULT 0;

   DECLARE cur1 CURSOR FOR SELECT emp_id,
emp_name, department, salary FROM employee;
DECLARE CONTINUE HANDLER FOR NOT FOUND
SET done = 1;
   OPEN cur1;
```

```
fetch_loop: LOOP
FETCH cur1 INTO cur_emp_id, cur_emp_name,
cur_department, cur_salary;
IF done THEN
LEAVE fetch_loop;
END IF;
 SELECT CONCAT('ID: ', cur_emp_id, ', Name: ',
cur_emp_name, ', Department: ', cur_department, ', Salary: ',
cur_salary) AS 'Employee Details';

   END LOOP fetch_loop;



CLOSE cur1;
END//
DELIMITER ;

CALL print_details();
```

OUTPUT:

Employee Details: ID: 1, Name: Sam, Department: Engineering, Salary: 45000

Employee Details: ID: 2, Name: Ram, Department: HR, Salary: 40000

Employee Details: ID: 3, Name: Pam, Department: IT, Salary: 55000

Employee Details: ID: 4, Name: John, Department: Engineering, Salary: 65000

Employee Details: ID: 5, Name: Mike, Department: IT, Salary: 48000

Employee Details: ID: 6, Name: Alice, Department: HR, Salary: 42000

Employee Details: ID: 7, Name: Bob, Department: Engineering, Salary: 50000

Employee Details:

```
ID: 8, Name: Charlie, Department: IT, Salary: 60000
```

Employee Details:

```
ID: 9, Name: David, Department: HR, Salary: 39000
```

Employee Details:

```
ID: 10, Name: Lewis, Department: Engineering, Salary: 70000
```

2. **Retrieve and display only employees with a salary greater than 50,000.**

```
DELIMITER //
CREATE PROCEDURE sal()
BEGIN

DECLARE cur_emp_id INT;
   DECLARE cur_emp_name VARCHAR(25);
   DECLARE cur_department VARCHAR(15);
   DECLARE cur_salary INT;
   DECLARE done INT DEFAULT 0;
```

```
    DECLARE cur1 CURSOR FOR SELECT emp_id,
emp_name, department, salary FROM employee WHERE
salary>50000;


    DECLARE CONTINUE HANDLER FOR NOT FOUND
SET done = 1;
OPEN cur1;


fetch_loop: LOOP
FETCH cur1 INTO cur_emp_id, cur_emp_name,
cur_department, cur_salary;
IF done THEN
LEAVE fetch_loop;
END IF;
 SELECT CONCAT('ID: ', cur_emp_id, ', Name: ',
cur_emp_name, ', Department: ', cur_department, ', Salary: ',
cur_salary) AS 'Employee Details whose salary > 50000';


    END LOOP fetch_loop;
CLOSE cur1;
END//
DELIMITER ;
```

CALL sal();

OUTPUT:

Employee Details whose salary > 50000:

ID: 3, Name: Pam, Department: IT, Salary: 55000

Employee Details whose salary > 50000:

ID: 4, Name: John, Department: Engineering, Salary: 65000

Employee Details whose salary > 50000:

ID: 8, Name: Charlie, Department: IT, Salary: 60000

Employee Details whose salary > 50000:

ID: 10, Name: Lewis, Department: Engineering, Salary: 70000

3. **Increase the salary of employees in the IT department by 10%.**

DELIMITER //

CREATE PROCEDURE increase_sal()

```
BEGIN

DECLARE cur_emp_id INT;
   DECLARE cur_emp_name VARCHAR(25);
   DECLARE cur_department VARCHAR(15);
   DECLARE cur_salary INT;
   DECLARE new_sal INT;
   DECLARE done INT DEFAULT 0;

   DECLARE cur3 CURSOR FOR SELECT emp_id,
emp_name, department, salary FROM employee WHERE
department='IT';

   DECLARE CONTINUE HANDLER FOR NOT FOUND
SET done = 1;
OPEN cur3;

fetch_loop: LOOP
FETCH cur3 INTO cur_emp_id, cur_emp_name,
cur_department, cur_salary;
IF done THEN
LEAVE fetch_loop;
END IF;
      SET new_sal = cur_salary*1.10;
```

```
        UPDATE employee SET salary = new_sal WHERE
emp_id=cur_emp_id;



    END LOOP fetch_loop;
CLOSE cur3;
END//
DELIMITER ;


CALL increase_sal();
SELECT * FROM employee;
```

OUTPUT:

emp_id	emp_name	department	salary
1	Sam	Engineering	45000
2	Ram	HR	40000
3	Pam	IT	60500
4	John	Engineering	65000
5	Mike	IT	52800
6	Alice	HR	42000
7	Bob	Engineering	50000
8	Charlie	IT	66000
9	David	HR	39000
10	Lewis	Engineering	70000

5. **Calculate and display the total salary per department using a cursor.**

```
DELIMITER //
CREATE PROCEDURE tot_sal()
```

```
BEGIN

    DECLARE cur_department VARCHAR(15);
    DECLARE tot_sal INT;
    DECLARE done INT DEFAULT 0;

    DECLARE cur5 CURSOR FOR SELECT department,
SUM(salary) FROM employee GROUP BY department;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
SET done = 1;
OPEN cur5;

fetch_loop: LOOP
FETCH cur5 INTO cur_department, tot_sal;
IF done THEN
LEAVE fetch_loop;
END IF;

SELECT CONCAT( 'Department: ', cur_department, ', Total
Salary: ', tot_sal ) AS 'Total Salary by Department';

    END LOOP fetch_loop;
CLOSE cur5;
```

```
END//
DELIMITER ;



CALL tot_sal();
```

OUTPUT:

Total Salary by Department
Department: Engineering, Total Salary: 230000

Total Salary by Department
Department: HR, Total Salary: 121000

6. **Pass a department name as a parameter and fetch only those employees.**

```
DELIMITER //
CREATE PROCEDURE dept_details(cur_department
VARCHAR(15))
BEGIN


DECLARE cur_emp_id INT;
   DECLARE cur_emp_name VARCHAR(25);
   DECLARE cur_salary INT;
   DECLARE done INT DEFAULT 0;
```

```
    DECLARE cur6 CURSOR FOR SELECT emp_id,
emp_name, salary FROM employee WHERE department =
cur_department ;
DECLARE CONTINUE HANDLER FOR NOT FOUND
SET done = 1;
    OPEN cur6;

fetch_loop: LOOP
FETCH cur6 INTO cur_emp_id, cur_emp_name, cur_salary;
IF done THEN
LEAVE fetch_loop;
END IF;
 SELECT CONCAT('ID: ', cur_emp_id, ', Name: ',
cur_emp_name, ', Department: ', cur_department, ', Salary: ',
cur_salary) AS 'Employee Details for specific department';

    END LOOP fetch_loop;

CLOSE cur6;
END//
DELIMITER ;

CALL dept_details('HR');
```

OUTPUT:

Employee Details for specific department:

```
ID: 2, Name: Ram, Department: HR, Salary: 40000
```

Employee Details for specific department:

```
ID: 6, Name: Alice, Department: HR, Salary: 42000
```

Employee Details for specific department:

```
ID: 9, Name: David, Department: HR, Salary: 39000
```

7. **Fetch data from one table and insert it into another using a cursor.**

```
DELIMITER //
CREATE PROCEDURE copy_data()
BEGIN

    DECLARE cur_emp_id INT;
    DECLARE cur_emp_name VARCHAR(25);
    DECLARE cur_department VARCHAR(15);
    DECLARE cur_salary INT;
```

```
DECLARE done INT DEFAULT 0;
DECLARE cur7 CURSOR FOR SELECT emp_id, emp_name, department, salary FROM employee;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
DROP TABLE IF EXISTS employee_copy;
CREATE TABLE employee_copy (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(25),
    department VARCHAR(15),
    salary INT
);



OPEN cur7;



fetch_loop: LOOP
    FETCH cur7 INTO cur_emp_id, cur_emp_name, cur_department, cur_salary;

    IF done THEN
        LEAVE fetch_loop;
```

```
        END IF;

        INSERT INTO employee_copy (emp_id, emp_name,
department, salary) VALUES (cur_emp_id, cur_emp_name,
cur_department, cur_salary);

    END LOOP fetch_loop;

    CLOSE cur7;
END//
DELIMITER ;

CALL copy_data();
SELECT * FROM employee_copy;
```

OUTPUT:

emp_id	emp_name	department	salary
1	Sam	Engineering	45000
2	Ram	HR	40000
4	John	Engineering	65000
6	Alice	HR	42000
7	Bob	Engineering	50000
9	David	HR	39000
10	Lewis	Engineering	70000
NULL	NULL	NULL	NULL

---

12. WORKING WITH PL/SQL TRIGGERS

1.-- Creating the employees table

CREATE TABLE employees (employee_id INT PRIMARY KEY, name VARCHAR(100), department VARCHAR(50), salary DECIMAL(10, 2));

-- Creating the employee_log table

CREATE TABLE employee_log (log_id INT AUTO_INCREMENT PRIMARY KEY, employee_id INT, operation VARCHAR(10), operation_time TIMESTAMP, FOREIGN KEY (employee_id) REFERENCES employees(employee_id));

-- Creating the trigger to log insert operations into employee_log DELIMITER //

CREATE TRIGGER log_employee_insert

AFTER INSERT ON employees

FOR EACH ROW

BEGIN

INSERT INTO employee_log (employee_id, operation, operation_time) VALUES (NEW.employee_id, 'INSERT', NOW());

END; //

DELIMITER ;

INSERT INTO employees

VALUES(23110, "Rohith","aids",10000);

SELECT * FROM employee_log;

OUTPUT:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

log_id	employee_id	operation	operation_time
1	23110	INSERT	2025-03-07 04:37:33
NULL	NULL	NULL	NULL

2. Prevent inserting or updating an employee's salary to a negative value.

DELIMITER //

CREATE TRIGGER check_salary_negative BEFORE INSERT ON employees FOR EACH ROW BEGIN -- Check if the salary is negative IF NEW.salary < 0 THEN -- Raise an error if salary is negative SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be negative'; END IF; END; //

DELIMITER ; INSERT INTO employees VALUES(345,"sr","aids",-9000)

OUTPUT:

Error Code: 1644. Salary cannot be negative

3. When a transaction occurs, record the previous balance in account_history.

-- Creating the 'accounts' table CREATE TABLE accounts (account_id INT PRIMARY KEY, account_name VARCHAR(100), balance DECIMAL(10, 2) -- Account balance);

-- Creating the 'account_history' table to store balance changes CREATE TABLE account_history (history_id INT AUTO_INCREMENT PRIMARY KEY, account_id INT, previous_balance DECIMAL(10, 2), change_time TIMESTAMP, FOREIGN KEY (account_id) REFERENCES accounts(account_id));

-- Creating the trigger to record the previous balance before an account update DELIMITER //

CREATE TRIGGER record_previous_balance BEFORE UPDATE ON accounts FOR EACH ROW BEGIN -- Check if the balance has changed IF OLD.balance <> NEW.balance THEN -- Insert a new record into account_history with the old balance and the current time INSERT INTO account_history (account_id, previous_balance, change_time) VALUES (OLD.account_id, OLD.balance, NOW()); END IF; END; //

DELIMITER ; INSERT INTO accounts (account_id, account_name, balance) VALUES (0, 'John Doe Account', 1000.00);

INSERT INTO accounts (account_id, account_name, balance) VALUES (2, 'Jane Smith Account', 500.00);

-- Sample Update to 'accounts' table (this will trigger the creation of records in 'account_history') UPDATE accounts SET balance = 1500.00 WHERE account_id = 1;

UPDATE accounts SET balance = 600.00 WHERE account_id = 2;

-- Check the contents of the 'account_history' table to see the previous balance log SELECT * FROM account_history;

OUTPUT:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

history_id	account_id	previous_balance	change_time
1	1	5000.00	2025-03-07 05:00:04
2	2	500.00	2025-03-07 05:00:04
NULL	NULL	NULL	NULL

4. Prevent deleting an admin user from the users table.

```
-- Creating the 'users' table
DROP TABLE IF EXISTS users;
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    username VARCHAR(100),
    role VARCHAR(50) -- role of the user (e.g., 'admin', 'user',
etc.)
);

-- Sample data insertion into the 'users' table
INSERT INTO users (user_id, username, role)
VALUES (1, 'admin_user', 'admin');

INSERT INTO users (user_id, username, role)
VALUES (2, 'regular_user', 'user');

-- Creating the trigger to prevent the deletion of an admin user
```

```
DELIMITER //

CREATE TRIGGER prevent_admin_deletion
BEFORE DELETE ON users
FOR EACH ROW
BEGIN
   IF OLD.role = 'admin' THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
'Cannot delete an admin user';
   END IF;
END; //

DELIMITER ;
-- Try to delete the admin user (this will raise an error)
DELETE FROM users WHERE user_id = 1;
```

5. Prevent customers from ordering more than 10 items at a time.

```
-- Drop the 'orders' table if it exists DROP TABLE IF EXISTS
orders;

-- Create the 'orders' table CREATE TABLE orders ( order_id
INT AUTO_INCREMENT PRIMARY KEY, customer_id
INT, product_id INT, quantity INT );

-- Insert sample data into the 'orders' table (valid orders with
quantity <= 10) INSERT INTO orders (customer_id,
product_id, quantity) VALUES (1, 101, 5);

INSERT INTO orders (customer_id, product_id, quantity)
VALUES (2, 102, 8);
```

-- Create the trigger to prevent large orders (quantity > 10) DELIMITER //

CREATE TRIGGER prevent_large_orders BEFORE INSERT ON orders FOR EACH ROW BEGIN -- Check if the quantity of the order is greater than 10 IF NEW.quantity > 10 THEN -- Raise an error and prevent the order from being inserted SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cannot order more than 10 items at a time'; END IF; END; //

DELIMITER ; -- Try inserting an order with quantity 15 (this should raise an error) INSERT INTO orders (customer_id, product_id, quantity) VALUES (3, 103, 15);

OUTPUT:

Error Code: 1644. Cannot order more than 10 items at a time

6. When a customer is deleted, log the action

-- Step 1: Drop the 'employee_log' table if it exists to remove any existing constraints DROP TABLE IF EXISTS employee_log;

-- Drop the 'employees' table if it exists (this should not raise any errors anymore) DROP TABLE IF EXISTS employees;

-- Create the 'employees' table CREATE TABLE employees (employee_id INT PRIMARY KEY, name VARCHAR(100), email VARCHAR(100));

-- Create the 'employee_log' table with a foreign key constraint CREATE TABLE employee_log (log_id INT AUTO_INCREMENT PRIMARY KEY, employee_id INT,

action VARCHAR(10), action_time TIMESTAMP, FOREIGN KEY (employee_id) REFERENCES employees(employee_id) ON DELETE CASCADE);

-- Creating the trigger to log delete operations into employee_log DELIMITER //

CREATE TRIGGER log_employee_deletion BEFORE DELETE ON employees FOR EACH ROW BEGIN -- Insert into the employee_log table when an employee is deleted INSERT INTO employee_log (employee_id, action, action_time) VALUES (OLD.employee_id, 'DELETE', NOW()); END; //

DELIMITER ;

-- Insert sample data into the 'employees' table INSERT INTO employees (employee_id, name, email) VALUES (101, 'John Doe', 'john.doe@example.com');

INSERT INTO employees (employee_id, name, email) VALUES (102, 'Jane Smith', 'jane.smith@example.com');

-- Step 5: Delete an employee to test the trigger DELETE FROM employees WHERE employee_id = 101;

-- Step 6: Check the contents of the 'employee_log' table to verify the log SELECT * FROM employee_log;

---

13. PROJECT REPORT

Aim:

The Pharmacy Management System will be a web-based application where users can enter specific details about medicines they are looking for and find nearby pharmacies that have them in stock.

Overview:

To build a Pharmacy Management System (PMS) designed to help users quickly find out where they can buy the medicines they need. The system uses a database to store important details like medicine names, available quantities, expiry dates, prices, and pharmacy locations.

Objectives:

- The system is built to cater to urgent scenarios, especially where quick access to medicine is critical.
- It enables patients and caregivers to quickly find required medicines in nearby pharmacies, eliminating the need to search multiple stores.
- By streamlining the medicine search process, it minimizes delays and reduces stress during emergencies.
- It enhances the overall process of locating and purchasing medications, making it more efficient and user-friendly.

Constraints:

- The system should handle a high number of concurrent users and allow pharmacy updates in real-time.

- The system must comply with local regulations regarding privacy and health-related data security.

Assumptions:

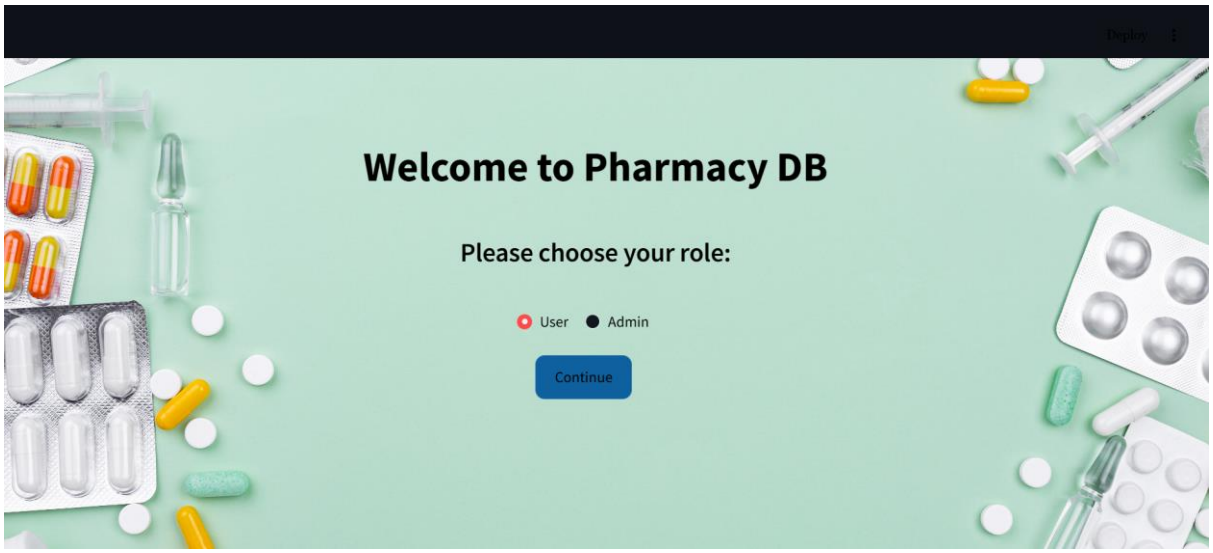
- The system should handle a high number of concurrent users and allow pharmacy updates in real-time.
- The system must comply with local regulations regarding privacy and health-related data security.

Screenshots:

Deploying the app using STREAMLIT

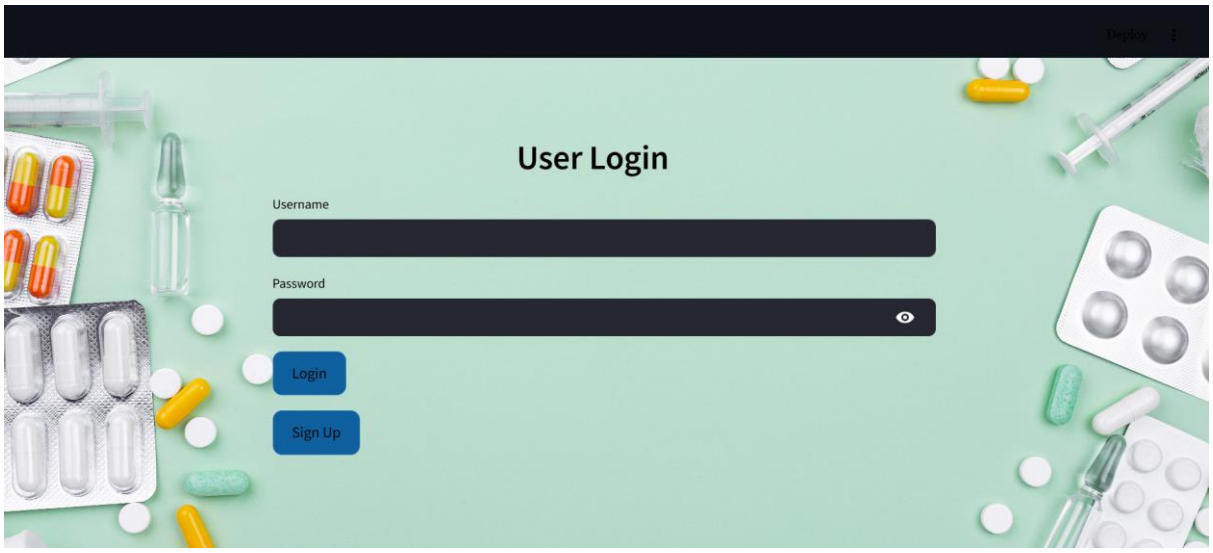
<u>HOME PAGE:</u>

The home page displayed requires the person accessing the system to choose their role.

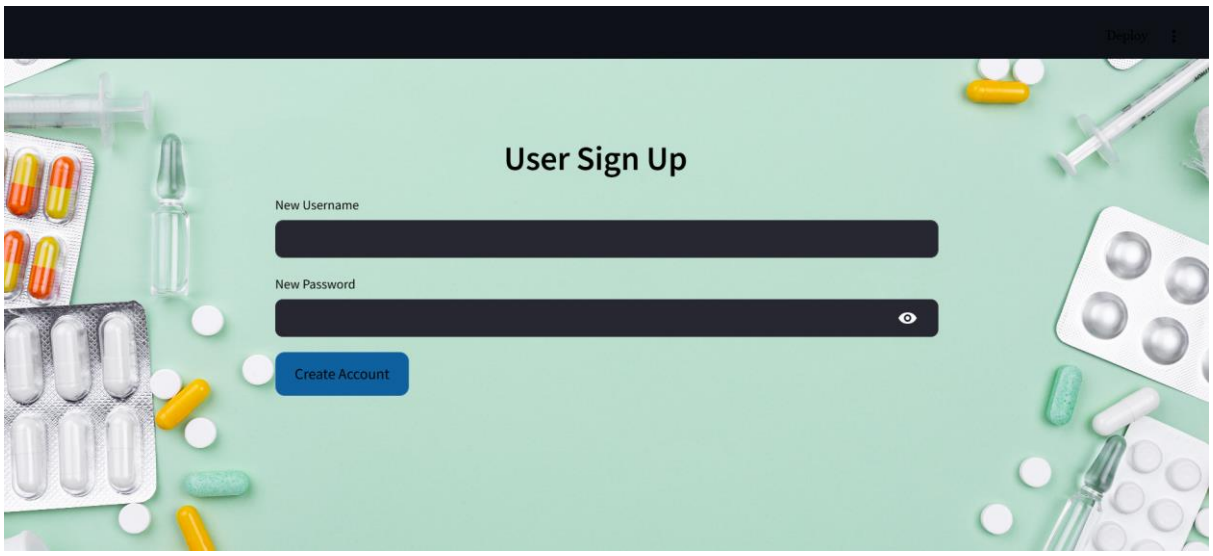


LOGIN PAGE:

Depending on your role, you would be directed to the Login page. If you are a new user, you can create an account and then gain access further into the system.

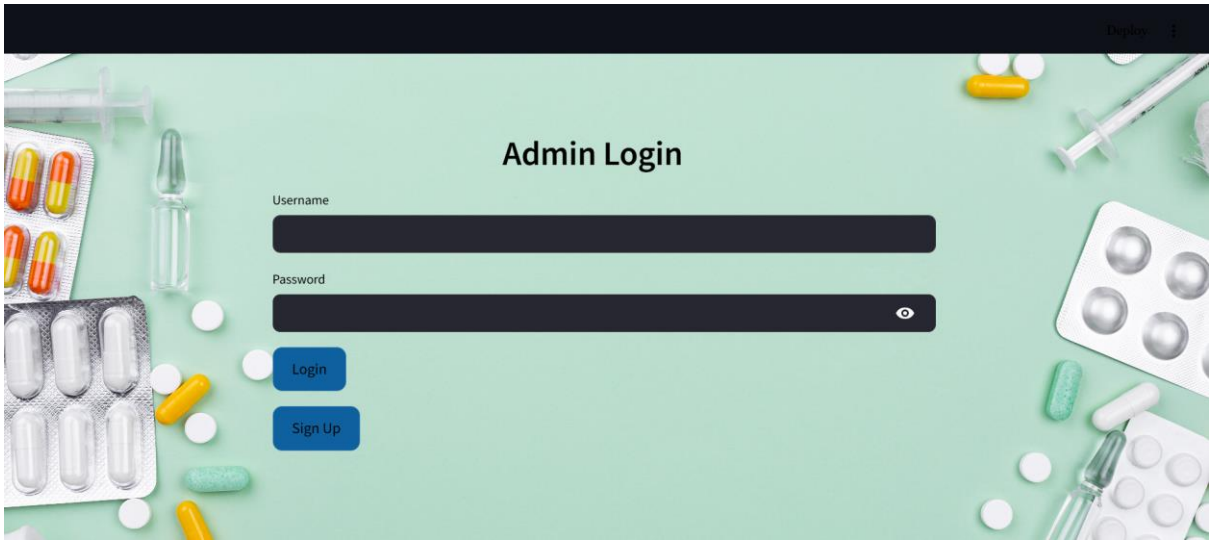


(user login)

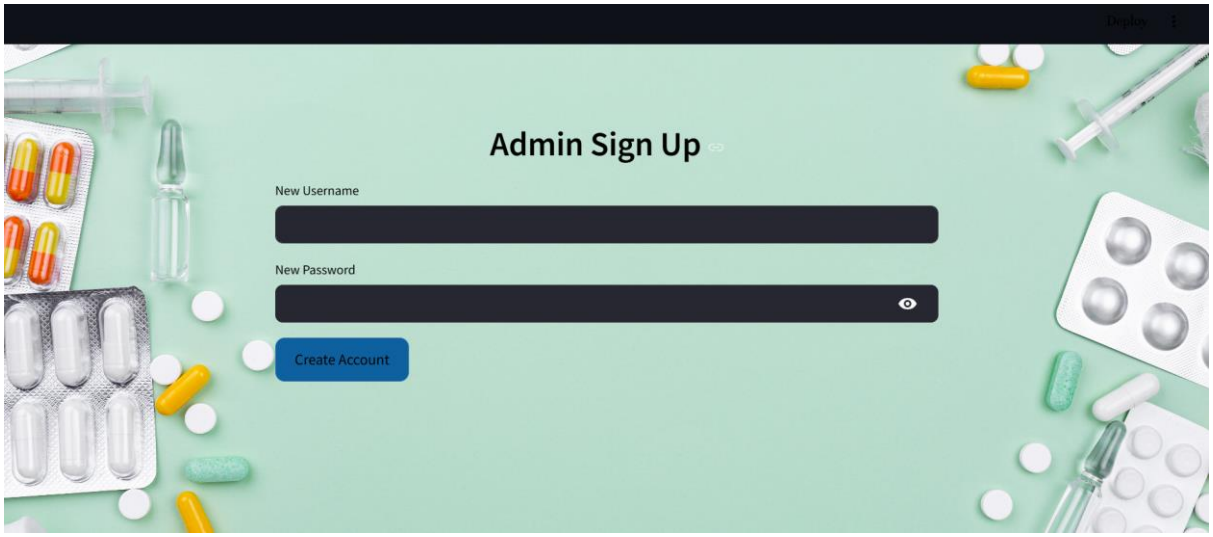


(user sign up)

Once the user has created an account, the page will be redirected back to the login page. The new credentials can be used and the user can log in.



(admin login)



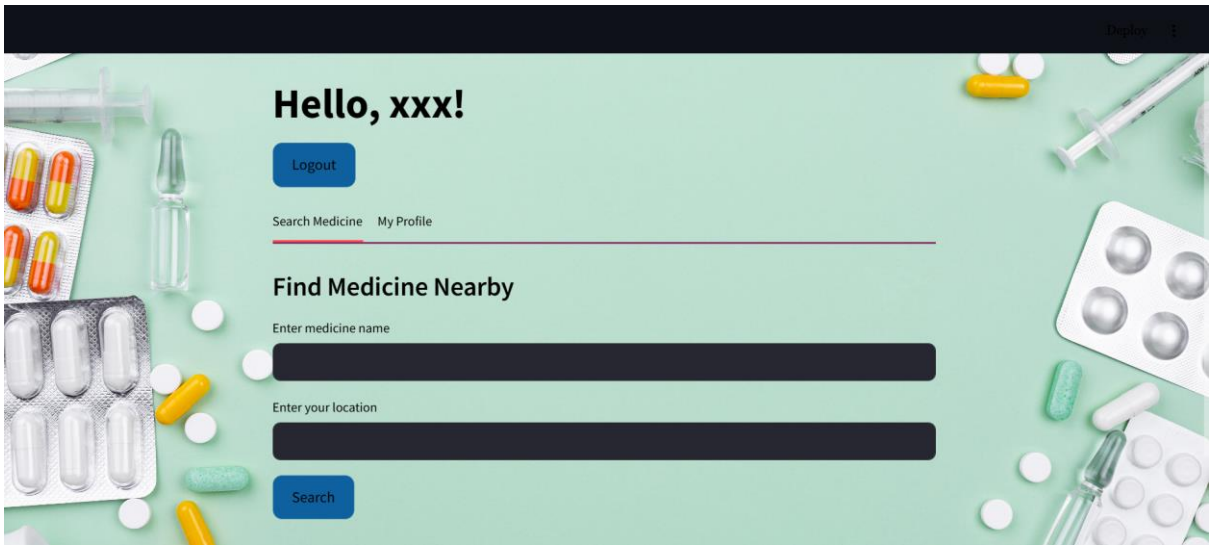
(admin sign up)

Once the admin has created an account, the page will be redirected back to the login page. The new credentials can be used and the admin can log in.

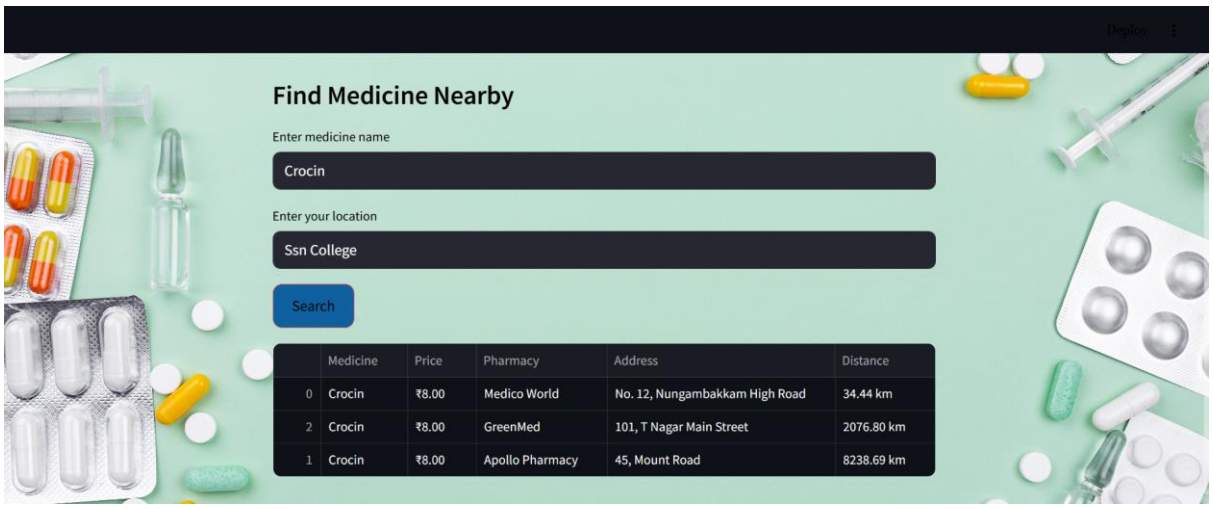
DASHBOARD:

For user:

The dashboard will allow the user to fill in the name of the medicine required and their respective location.

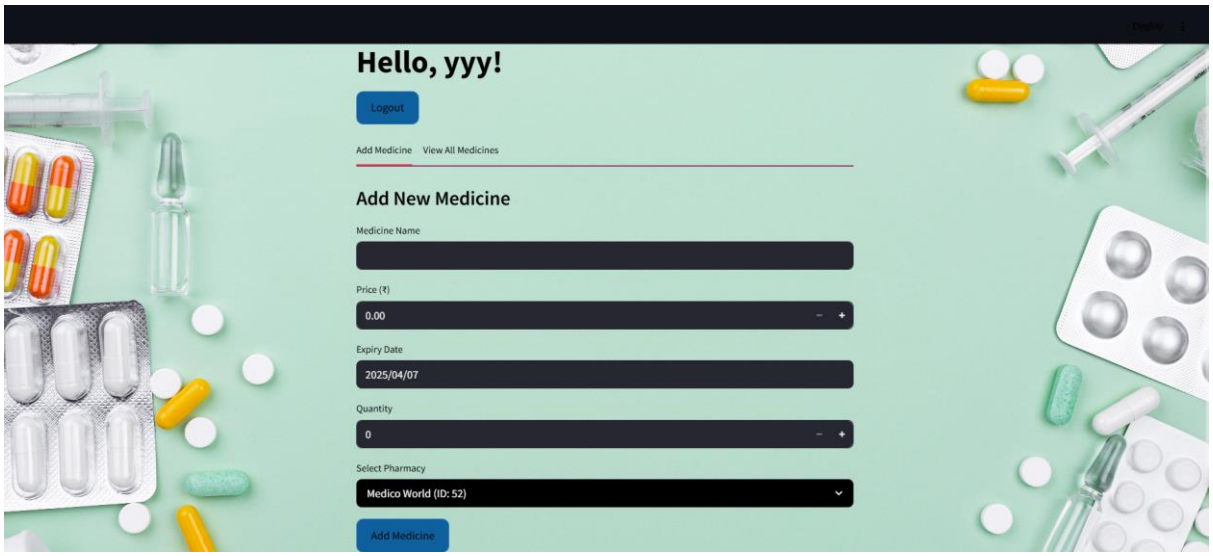


Once the search button is clicked, the pharmacies nearer to the user's location is listed in ascending order, along with their addresses.

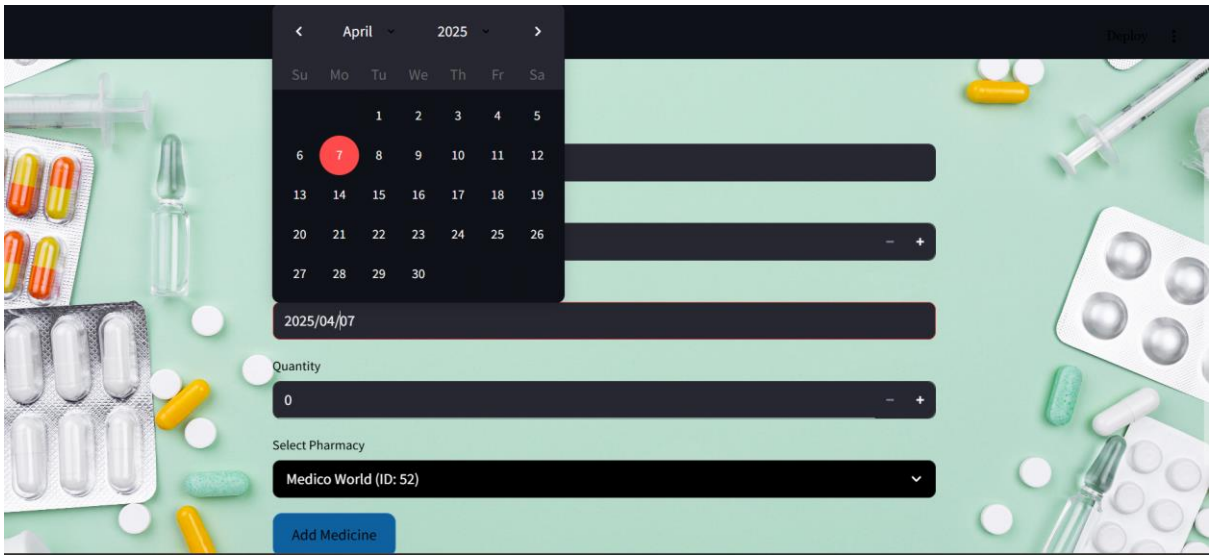


For admin:

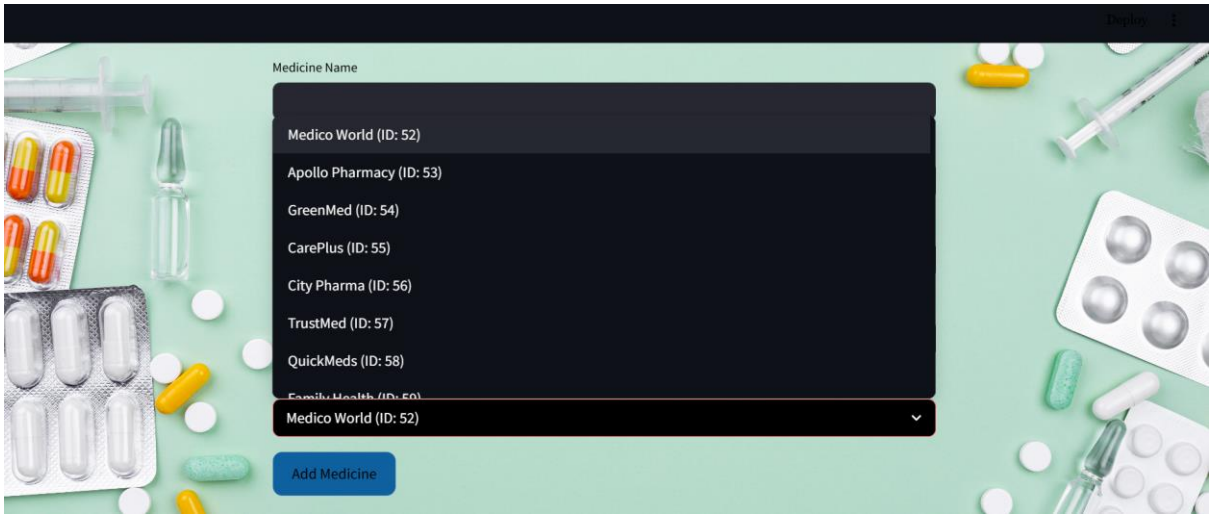
The dashboard will allow the admin to add a medicine to a particular pharmacy.



- The admin can allocate the price and quantity for a particular medicine.

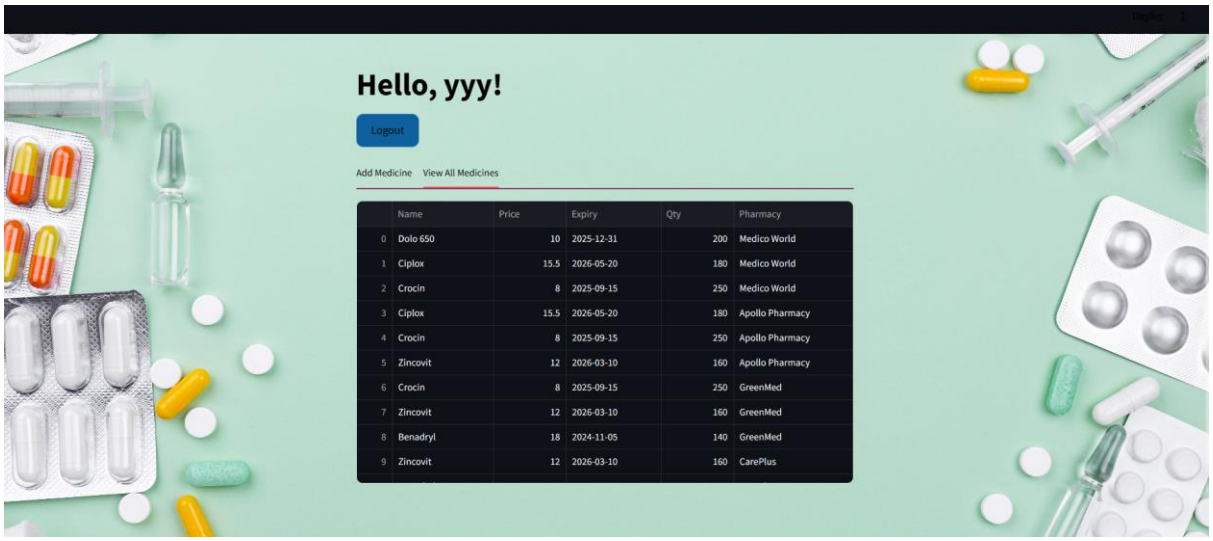


- The expiry date can be chosen from the calendar drop down or typed into the text box.



- All the available pharmacies will be listed in the drop-down and the admin must choose to pharmacy to which the medicine must be added.
- Once the medicine is added, a ‘successfully added’ message would be displayed.
- It can be checked if the medicine is added in the ‘View all medicines’ menu.

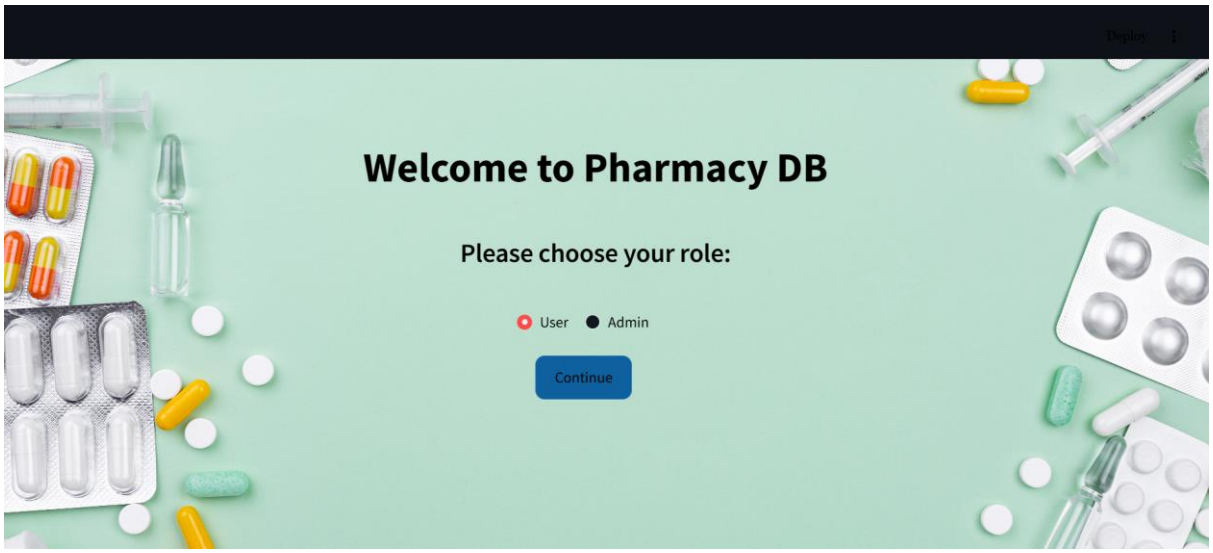
View all medicines:



It also allows the admin to view all the medicines available in all the pharmacies of the locality.

LOGOUT:

After clicking on logout, you will be redirected back to the home page.



CONCLUSION:

This system is designed to meet the urgent needs of users, particularly in emergency situations where quick access to medicine is critical. For patients and caregivers, the system provides a fast way to locate medicines in nearby pharmacies, saving valuable time that would otherwise be spent searching multiple stores.